



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

The four components of PEAS are:

1. Performance Measure – Defines how the performance of the intelligent agent is evaluated.

2. Environment – The world in which the agent operates

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

The variables that represent the physical state of the environment include width, height, agent pos, walls, food positions, opponents, score, steps, and collision. After extending the environment, toxic traps also represents part of the environment state because it stores the locations of the toxic traps.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The environment is a Multi-Agent Environment because it contains opponents in `self.opponents`. The opponents can independently move within the environment and can interact with the main agent. Therefore, the environment involves more than one active agent.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all percepts that an agent has received from its sensors over time. It represents the information that the agent has perceived from the environment throughout its operation.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The `get_percept()` function represents the Sensors component of PEAS. It provides the agent with information about the current environment, such as its position, opponent positions, food, walls, collisions, score, remaining food, and toxic trap information.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The baseline environment is fully observable because `get_percept()` provides the agent with the relevant state information, including the agent's position, opponent positions, food locations through the food percept, wall information, collision status, score, and remaining food. Therefore, the agent has sufficient information to determine the current state of the environment.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance Measure component is updated. When the agent attempts to move into a wall, the environment deducts 5 points from `self.score`. The score represents the performance measure used to evaluate how well the agent performs.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Environment rather than the agent's internal processing effort. An agent may perform a large number of calculations without achieving a useful result. By measuring external outcomes, such as collecting food, avoiding collisions, and avoiding penalties, the metric encourages the agent to take actions that improve its actual performance in the environment.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

- Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the

environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment becomes Partially Observable. Although the toxic traps exist in the physical environment, the agent cannot directly perceive their locations through its sensors. Therefore, the agent does not have complete information about the current environment state and must make decisions with incomplete

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

The `smells_toxin` sensor provides the agent with information about whether its current location contains a toxic trap. This information allows a rational agent to avoid actions that may lead to a toxic trap and cause a 15-point penalty. Therefore, the sensor helps the agent make better decisions based on the

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

A classic example is a Vacuum World where the agent receives a reward every time it performs the Suck action, even when the current square is already clean. The agent could repeatedly perform Suck on a clean square to increase its score without actually improving the cleanliness of the environment. This

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING